

Exemple d'affichage complet

📌 Objectifs

- Synthétiser les éléments clés d'un chapitre.
- Illustrer avec un extrait de code concis.
- Donner 1 à 2 repères visuels (tableau, schéma).

💡 Définitions

Algorithme : suite finie d'instructions permettant de résoudre un problème.

Complexité : quantité de ressources (temps, mémoire) consommées par un algorithme.

🐍 Code Python (basique)

```
root@python:~$
def somme(liste):
    total = 0
    for x in liste:
        total += x
    return total

print(somme([1, 2, 3])) # 6
```

⚙️ Idiomes Python

```
root@python:~$
# Compréhension de liste
evens = [x for x in range(10) if x % 2 == 0]
```


Slicing

```
s = "NSI"  
print(s[::-1]) # ISN
```

Tableau compact

Structure Mutable		Clés/Index
list	Oui	Index entiers
tuple	Non	Index entiers
dict	Oui	Clés hachables

Schéma (image)



NSI
4ISM


```
def decrypt(key):
```

```
print()
```



Note

Astuce : préférez plusieurs petites sections à une seule très dense.

Voir aussi : [Documentation Python](#).

Exemple de mise en page A4 optimisée

FOLLOW
THE
PYTHONIC
WAY

HACK
AGAINST
MACHISM

Les chaînes de caractères en Python

■ Définitions & Propriétés

- Une chaîne est une suite de caractères entre guillemets.
- Type immuable : on ne peut pas modifier une chaîne après sa création.

```
root@python:~$  
mot = "Python"  
print(mot[0]) # P  
mot[0] = "J"   # Erreur !
```

+ Opérations de base

```
root@python:~$  
s = "NSI"  
print(s + " Python")  
print(s * 3)  
print(s[1])
```

Fiche NSI – Chaînes de caractères – Terminale

FOLLOW
THE
PYTHONIC
WAY

HACK
AGAINST
MACHISM

Les Listes en Python



Définition et Création

Une liste est une collection de données ordonnée et modifiable. C'est l'une des structures de données les plus importantes en Python. Elle est reconnaissable par les crochets [] qui l'entourent. On peut y stocker des éléments de types différents (nombres, chaînes de caractères, booléens, etc.).

```
root@python:~$
# Création une liste vide
ma_liste = []
# Création une liste avec des éléments
fruits = ["pomme", "banane", "orange"]
# Une liste peut contenir des types différents
melange = ["Python", 2024, True, 3.14]
# Obtenir la longueur d'une liste avec len()
longueur = len(fruits)
print(f"La liste 'fruits' contient {longueur} éléments.") # La liste 'fruits' c
```

La compréhension de liste (List Comprehension)

C'est une syntaxe rapide et très "pythonique" pour créer des listes à partir d'autres collections. Elle combine une boucle et une condition sur une seule ligne. C'est une astuce de Python que vous rencontrerez souvent.

```
root@python:~$
# Créer une liste des carrés des nombres pairs de 0 à 10
carres_pairs = [x*x for x in range(11) if x % 2 == 0]
print(carres_pairs) # Affiche [0, 4, 16, 36, 64, 100]
```



Indexation et Accès aux éléments

Chaque élément d'une liste possède une position, appelée index. En Python, l'indexation commence à 0.

On peut aussi utiliser des index négatifs pour accéder aux éléments à partir de la fin de la liste. L'index -1 correspond au dernier élément, -2 à l'avant-dernier, et ainsi de suite.


```
root@python:~$
animaux = ["chien", "chat", "oiseau", "poisson"]
# Indexation positive
print(animaux[0]) # Affiche "chien"
print(animaux[2]) # Affiche "oiseau"
# Indexation négative
print(animaux[-1]) # Affiche "poisson"
print(animaux[-3]) # Affiche "chat"
# Le slicing : Extraire une partie de la liste [début:fin] : La fin est exclue
print(animaux[1:3]) # Affiche ['chat', 'oiseau']
print(animaux[2:]) # Affiche ['oiseau', 'poisson']
```

⚙️ Modification de la liste

Les listes sont mutables, ce qui signifie qu'on peut ajouter ou supprimer des éléments après leur création.

Ajouter des éléments

- `append(element)`: ajoute un élément à la fin de la liste.
- `insert(index, element)`: insère un élément à un index précis.
- `extend(iterable)`: ajoute tous les éléments d'un autre objet (comme une autre liste) à la fin.

```
root@python:~$
jours = ["lundi", "mardi", "jeudi"]
jours.append("vendredi")
print(jours) # ['lundi', 'mardi', 'jeudi', 'vendredi']
jours.insert(2, "mercredi")
print(jours) # ['lundi', 'mardi', 'mercredi', 'jeudi', 'vendredi']
week_end = ["samedi", "dimanche"]
jours.extend(week_end)
print(jours) # ['lundi', 'mardi', 'mercredi', 'jeudi', 'vendredi', 'samedi',
```

Supprimer des éléments

- `del ma_liste[index]`: supprime l'élément à l'index donné.

```
root@python:~$
nombres = [10, 20, 30, 40]
# Supprimer élément à index 1 (le nombre 20)
del nombres[1]
print(nombres) # [10, 30, 40]
```

- `pop(index)` : supprime l'élément à l'index donné et le renvoie. Si l'index est omis, il supprime et renvoie le dernier élément.

```
root@python:~$
nombres = [10, 20, 30, 40]
# Supprimer le dernier élément et le stocker
element_supprime = nombres.pop()
print(f"Élément supprimé : {element_supprime}") # Élément supprimé : 40
print(nombres)    # [10, 30]
```

Parcourir une liste

Une boucle `for` est l'outil principal pour parcourir les éléments d'une liste.

1. Par élément :

C'est la méthode la plus simple et la plus courante. On parcourt directement chaque valeur de la liste.

```
root@python:~$
pizzas = ["reine", "calzone", "pepperoni"]
for pizza in pizzas:
    print(pizza)
```

Affiche :

```
reine
calzone
pepperoni
```

2. Par index :

Utile si vous avez besoin de la position de l'élément en plus de sa valeur. On utilise la fonction `range(len(liste))`.

*FOLLOW
THE
PYTHONIC
WAY*

*#HACK
AGAINST
MACHISM*


```
root@python:~$
pizzas = ["reine", "calzone", "pepperoni"]
for i in range(len(pizzas)):
    print(f"Pizza n°{i} : {pizzas[i]}")
```

Affiche :

Pizza n°0 : reine
Pizza n°1 : calzone
Pizza n°2 : pepperoni

3. Parcours mixte (avec enumerate) :

La méthode la plus élégante pour avoir à la fois l'index et l'élément. Elle est à privilégier par rapport à la méthode précédente.

```
root@python:~$
pizzas = ["reine", "calzone", "pepperoni"]
for index, pizza in enumerate(pizzas):
    print(f"Index {index} correspond à la {pizza}.")
```

🤔 Copier une liste

Affiche :

Index 0 correspond à la reine.
Index 1 correspond à la calzone.
Index 2 correspond à la pepperoni.

```
root@python:~$
liste_a = [1, 2, 3]
liste_b = liste_a # C'est une référence, pas une copie !
liste_b.append(4)
print(liste_a) # Affiche [1, 2, 3, 4] !
```

Pour faire une vraie copie, on peut utiliser la technique du slicing ou la méthode `.copy()`.

```
root@python:~$
liste_a = [1, 2, 3]
liste_c = liste_a[:] # Vraie copie avec slicing
liste_c.append(4)
print(liste_a) # Affiche [1, 2, 3]
print(liste_c) # Affiche [1, 2, 3, 4]
```


Les Structures de Données

Les Listes (list)

- Collection ordonnée et modifiable d'éléments.
- Permet les doublons.

```
root@python:~$  
ma_liste = [1, "a", 3.14, 1]  
ma_liste.append(5)  
print(ma_liste[1]) # a
```

Les Dictionnaires (dict)

Collection non-ordonnée de paires clé-valeur. Mutable.

```
root@python:~$  
personne = {"nom": "Alan", "age": 41}  
print(personne["nom"]) # Alan
```

*FOLLOW
THE
PYTHONIC
WAY*

*HACK
AGAINST
MACHISM*